Oracle Course Automator

Internal automation tool for **AGSM AIM** that automates repetitive data-entry workflows in **Oracle HCM Cloud** (Learning / "Apprendimento" module).

A member of the training team opens a web page, provides the data (via a form, an Excel file, or a natural-language sentence), and the tool drives a browser through Oracle to perform the work that was previously done by hand.

Table of contents

What it does

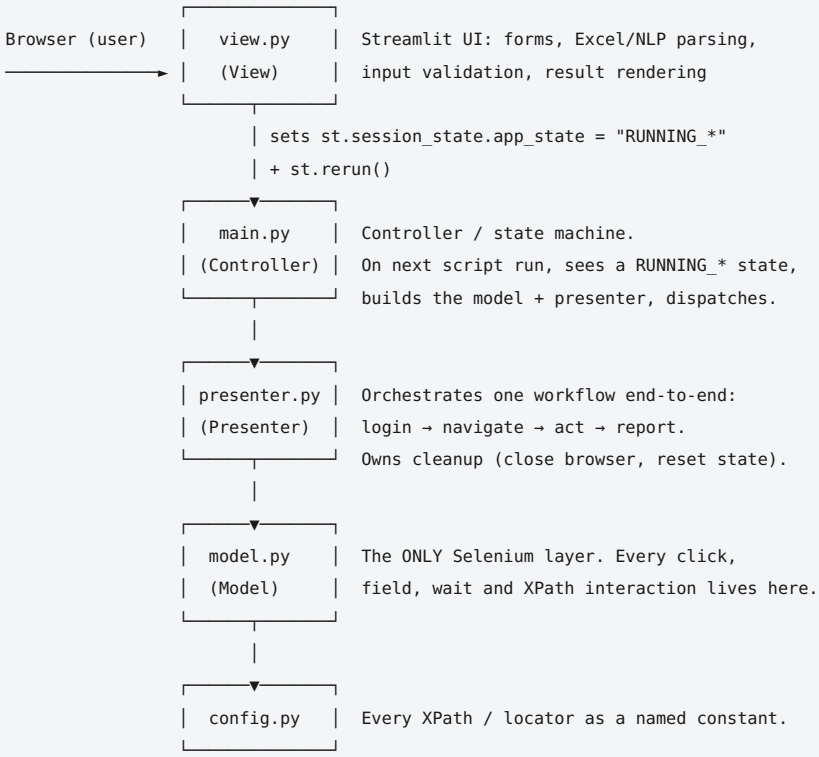
The tool covers four Oracle HCM Learning workflows:

#	Workflow	Oracle equivalent (manual)
1	**Course creation**	Create a course ("Corso")
2	**Edition + activities**	Create an edition ("Edizione") of a course and its daily activities ("Attività")
3	**Student enrollment**	Add learners ("Allievi") to an edition from a person-number list
4	**Presence assignment**	Mark completion status ("Assegnazione presenza") for enrolled learners

Each workflow accepts input in three ways: a **structured form**, an **Excel upload** (for bulk / batch operations), or a **natural-language sentence** (Italian) parsed with spaCy.

How it works (architecture)

The application follows the **MVP (Model-View-Presenter)** pattern, plus a thin controller/state-machine in `main.py`.



The single most important property to understand: the automation runs **synchronously inside a Streamlit script run**. There is no background thread and no job queue. When the user clicks a button, the View sets a state flag and triggers a rerun; on that next run, `main.py` opens the browser and blocks inside the Presenter until the whole workflow finishes.

This one fact explains several design decisions and limitations below (the WebSocket-timeout dependency, the one-at-a-time lock, and the "do not reload the page" warnings).

Request lifecycle (concrete example: create a single course)

1. User fills the course form and clicks **Crea Corso**.
2. `view.py` validates input, stores it in `st.session_state.course_details`, sets `app_state = "RUNNING_COURSE"`, and calls `st.rerun()`.
3. On the new script run, `main.py` sees a non-IDLE state and a per-session `automation_in_progress` guard that is not yet set, so it instantiates `OracleAutomator` (this opens Edge) and a `CoursePresenter`.
4. `main.py` dispatches to `presenter.run_create_course(...)`.
5. The presenter logs in, navigates to the Corsi page, checks whether the course already exists, creates it, and shows a result message.
6. The presenter's `finally` block closes the browser, releases the VM lock, sets `app_state = "IDLE"`, and reruns — returning the UI to its idle state.

Technology stack

Layer	Technology
Language	Python 3
Web UI	Streamlit
Browser automation	Selenium (Microsoft Edge + `msedgedriver`)
NLP (Italian)	spaCy (`it_core_news_sm`) + `spacy.Matcher` + regex fallback
Spreadsheet parsing	pandas + openpyxl
Date handling	python-dateutil

Target browser is **Microsoft Edge**, because that is what is installed and managed on the corporate Windows Server. The

Selenium driver `msedgedriver.exe` must match the installed Edge version (see Deployment).

Project layout

```
MVP_Selenium_Streamlit/
├── main.py               # Controller / state machine + logging setup + orphan cleanup
├── view.py               # Streamlit UI, input parsing (form/Excel/NLP), validation
├── presenter.py          # Workflow orchestration + cleanup
├── model.py              # Selenium layer (all browser interaction)
├── config.py             # All XPaths / locators as named constants
├── automation_lock.py    # VM-global one-at-a-time lock (file-based) + heartbeat
├── themes.json           # UI colour themes and fonts
├── user_preferences.json # Last-saved theme/font (written by the UI)
├── logo-agsm.jpg         # Logo shown in the UI
├── requirements.txt      # Python dependencies
├── packages.txt          # (legacy, unused on the Windows VM — see note below)
├── .streamlit/
│   ├── secrets.toml      # ORACLE_URL + EDGE_DRIVER_PATH  (NOT in git)
│   └── config.toml       # Streamlit server/theme config
├── logs/                 # session_YYYYMMDD.log  (created at runtime)
└── start_app.bat         # Windows launch script (prod)
```

> **Note on `packages.txt` :** it lists `chromium` / `chromium-driver` and is a leftover from an earlier Streamlit Community Cloud configuration. It has **no effect** on the Windows Server deployment, which uses Edge. It is harmless but can be deleted.

Runtime-generated folders — `logs/`, `__pycache__/`, and the virtual environment — are not part of the source and should be git-ignored.

Local development setup

Development is done on macOS (PyCharm); production is a Windows Server VM. The code is cross-platform (the `msedgedriver` path differs per machine and is read from `secrets.toml`).

```
# 1. Clone the repository
git clone <repo-url>
cd MVP_Selenium_Streamlit

# 2. Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate        # macOS/Linux
# .\venv\Scripts\activate       # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Download the Italian spaCy model (required, not in requirements.txt)
python -m spacy download it_core_news_sm

# 5. Provide msedgedriver matching your local Edge version
#    (see Configuration below)

# 6. Create .streamlit/secrets.toml (see Configuration below)
```

Configuration

Two secrets are read from `.streamlit/secrets.toml`. **This file is never committed to git** (it contains the production Oracle URL) — keep it in `.gitignore`.

```
# .streamlit/secrets.toml
ORACLE_URL = "https://<oracle-hcm-host>/hcmUI/faces/FuseWelcome"

# Path to the Edge WebDriver executable on THIS machine.
# On the server the driver sits next to the code; on Mac it is elsewhere.
# The first three version numbers of the driver MUST match installed Edge.
EDGE_DRIVER_PATH = "msedgedriver.exe"
```

Optional Streamlit configuration lives in `.streamlit/config.toml` (server address/port, theme). See Deployment for the production values.

Oracle credentials are NOT stored anywhere. Each user types their own Oracle username and password into the login screen at the start of a session; they are held only in `st.session_state` (server memory) for the duration of that session and are never written to disk.

Running the app

```
streamlit run main.py
```

Then open the URL Streamlit prints (locally `http://localhost:8501`).

On the production server the app is started with `start_app.bat` — see Deployment.

The four workflows

Each is available as its own tab in the UI, and each supports three input methods.

1. Course creation (`Creazione Corso`)

Creates a course. Batch mode (Excel) creates many courses in one run and skips any that already exist.

2. Edition + activities (`Creazione Edizione + Attività`)

Creates an edition of an existing course, fills its details (dates, location, supplier, price, and the "Attributi Aggiuntivi" fields), then creates each daily activity. Activity times are normalised to Oracle's required `HH.MM` format automatically. Batch mode creates many editions from one Excel file.

3. Student enrollment (`Aggiungi Allievi`)

Adds learners to an edition from a list of person numbers (TXT upload, Excel `ALLIEVI` sheet, or NLP). Batch mode processes multiple editions in one run.

4. Presence assignment (`Assegnazione Presenza`)

Marks a completion status (`Completato` / `Esente` / `Non passato`) for each learner's activities. Activities whose date is still in the future are skipped and reported separately (Oracle rejects a completion date in the future).

Key design decisions

These are the non-obvious choices a maintainer needs to know before changing anything.

All locators live in `config.py`

Every XPath is a named constant. When Oracle changes its UI (which it does on updates), you edit `config.py`, not the workflow logic. Most fields have multiple fallback XPaths tried in order.

Oracle ADF quirks are handled deliberately

Oracle HCM is built on Oracle ADF, which has behaviours that break naive Selenium code: - **Glass panes:** ADF shows a blocking overlay (`AFBlockingGlassPane` / `AFModalGlassPane`) during partial-page refreshes. The code waits for these to clear *before* reading results or clicking, otherwise clicks land on a stale or covered element. - **JavaScript `.value` does not fire ADF events.** Setting a field's value via JS updates the DOM but not ADF's client model, so Oracle sees an empty field. Date fields are set and then given explicit `input` + `change` events. - **Time format is `HH.MM` , not `HH:MM` .** Oracle rejects colons. A single `normalize_time()` function in `model.py` converts every time input (form, Excel, NLP) to `HH.MM` before it reaches Oracle.

Honest failure reporting

Where earlier versions reported a false "Successo", the code now detects the actual Oracle outcome and reports the real reason. Examples: an activity rejected because its date falls outside the offer window is reported as failed with Oracle's own message; learners skipped for a future date are listed separately rather than silently dropped.

Stale-element resilience

ADF re-renders regions after edits (e.g. tabbing out of a date field triggers a server-side validation refresh). A `_click_when_ready()` helper clears glass panes, retries on `StaleElementReferenceException` , falls back to a JS click, and reports the real exception on final failure. Critical multi-step buttons use it.

VM-global one-at-a-time lock (`automation_lock.py`)

Because one `streamlit run` process serves every colleague and the browser automation is synchronous, two simultaneous runs would fight over one browser. `automation_lock.py` is a **file-based** lock (visible to every session on the VM) with a heartbeat: a long healthy batch keeps proving it is alive, while a genuinely hung run (no progress for 8 minutes) can be reclaimed and its orphaned driver killed by PID. See Known limitations for its current activation status.

Per-session double-launch guard

`st.session_state.automation_in_progress` prevents a Streamlit rerun (e.g. from a reconnect) from starting a second browser within the same session.

Logging

- One file per day: `logs/session_YYYYMMDD.log` .
- Every line is stamped with the logged-in Oracle username (`user=<username>`), required by Sistemi Informativi for audit/traceability.
- `print()` is redirected so that all the workflow's progress messages also land in the log — the log is the primary tool for diagnosing a failed run.

Deployment

Full operational detail is in `DEPLOYMENT.md` . In brief:

- Runs on a **Windows Server VM**, started manually with `start_app.bat` , using a **portable Python** at `C:\Prod\python_portable` .
- Uses **Microsoft Edge** on the server; `msedgedriver.exe` sits next to the code and its version must match Edge (first three version numbers).
- Accessed by colleagues through a reverse proxy at `https://oraclecourseautomator.gruppomagis.it` .
- The reverse proxy's connection/idle timeout must be **generous (30 min)** because automations run synchronously and long batches must not be cut — see Known limitations.

Known limitations

Full detail, with the reasoning behind each, is in `LIMITAZIONI.md` . The three that matter most for a maintainer:

1. **Streamlit synchronous model.** A WebSocket reconnect during a long run can abandon the automation on script rerun. This is a platform property, not a fixable bug; it is *designed around* (proxy timeout, one-at-a-time use, "do not reload" warnings), not fixed.

2. **Edge / driver version coupling.** If Edge auto-updates and the driver does not, the browser will not start. Edge auto-update is disabled on the server; the driver must be updated in the same maintenance window when Edge is updated. This is an **operational dependency with a named owner** — see `DEPLOYMENT.md`.
3. **Oracle UI coupling.** The tool depends on Oracle's current DOM/XPaths. An Oracle update can change locators; fixes are localised to `config.py`.

Troubleshooting for developers

Symptom	Likely cause	Where to look
Browser does not start at all	Edge/driver version mismatch	Compare `msedgedriver --version` with Edge version; `EDGE_DRIVER_PATH` in `secrets.toml`
App works on the server console but drops via the company link	Reverse-proxy WebSocket/idle timeout too short	Proxy timeout config (30 min); `LIMITAZIONI.md`
A workflow fails at a specific field/step	Oracle changed a locator	The relevant XPath constant in `config.py`; the daily log
"Successivo" / a button "not found" but closes fast	ADF re-render made the element stale	Confirm the step uses `_click_when_ready`; read the real exception now in the log
Wrong "Successo" or silent skips	Result not verified against Oracle's response	The workflow's result-building code in `model.py` / `presenter.py`
Two browsers spawn / "occupato" confusion	Lock activation state	`automation_lock.py` and its call sites; `LIMITAZIONI.md`

When diagnosing any failed run, **start with the daily log** in `logs/` — it is timestamped, per-user, and contains the full step-by-step trace.
